Introduction to Optimization

CO 250

Kevin Shu

Lecture 4

Live (interactive) version of these slides: https://ootks.github.io/co250-lectures/lecture4.html

Lecture Outline

- Graph problems review
- Nonlinear Programming
- Return to Linear Programming

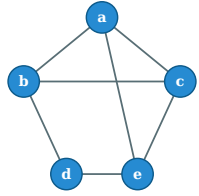
Graphs

Graphs

A **graph** $G = (V, E)$ consists of a set of 'vertices' V and a set of 'edges' E.

Formally, an edge is an unordered pair $\{i, j\}$, where i and j are in the vertex set.

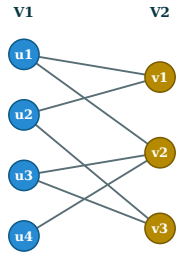
Graphs can be visualized as a picture where the vertices are points, and the edges are lines.



Bipartite Matching

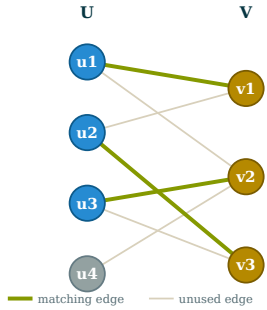
Bipartite Matching

A **bipartite graph** is a graph whose vertex set can be divided into two disjoint parts $V = V_1 \cup V_2$ with $V_1 \cap V_2 = \varnothing$, and every edge $e \in E$ consists of one element from V_1 and one element from V_2.



Bipartite Matching

A matching in a graph is a collection of edges in that graph so that no two edges in the collection share a vertex.



A maximum matching is a matching with at least as many edges as any other matching. A perfect matching is a matching where every vertex has a matched vertex.

Bipartite Matching: An ILP Formulation

How can we model maximum bipartite matching as an integer program?

Bipartite Matching: Decision variables

What should the variables be?

For each edge, we need to decide whether it is **part of the matching** or not.

For each edge $\{i, j\}$, introduce a variable x_{ij} that is 1 if that edge is in the matching and 0 otherwise.

Bipartite Matching: Constraints

Variables correspond to edges, constraints correspond to vertices.

For every vertex, there can be at most one incident edge. For each $i \in V$,

$$\sum_{j:\{i,j\}\in E} x_{ij} \leq 1.$$

Bipartite Matching: Objective

We want as many edges in the matching as possible.

To **count the number of edges in the matching, sum up the indicator variables.**

$$\max \sum_{\{i,j\} \in E} x_{ij}$$

Bipartite Matching: Assembling

$$\begin{array}{ll} \max & \sum_{\{i,j\} \in E} x_{ij} \\ \text{such that} & \sum_{j:\{i,j\} \in E} x_{ij} \leq 1 \quad \text{for each } i \in V \\ & 0 \leq x_{ij} \leq 1 \\ & x_{ij} \in \mathbb{Z} \end{array}$$

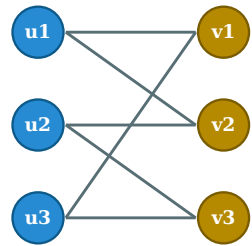
It turns out that the integrality constraints are not necessary! Matching can be solved with an LP.

Bipartite Matching: A Concrete Example

Consider this bipartite graph with parts $\{u_1, u_2, u_3\}$ and $\{v_1, v_2, v_3\}$.
Its edges are

$$E = \{\, u_1v_1,\ u_1v_2,\ u_2v_2,\ u_2v_3,\ u_3v_1,\ u_3v_3 \,\}.$$

We write x_{ij} for the variable of edge u_iv_j, so there are six variables.



Bipartite Matching: A Concrete Example

One constraint per vertex says each vertex is used at most once:

$$
\begin{array}{lll}
\max & x_{11} + x_{12} + x_{22} + x_{23} + x_{31} + x_{33} & \\
\text{such that} & x_{11} + x_{12} \leq 1 & (u_1) \\
& x_{22} + x_{23} \leq 1 & (u_2) \\
& x_{31} + x_{33} \leq 1 & (u_3) \\
& x_{11} + x_{31} \leq 1 & (v_1) \\
& x_{12} + x_{22} \leq 1 & (v_2) \\
& x_{23} + x_{33} \leq 1 & (v_3) \\
& 0 \leq x_{ij} \leq 1, \quad x_{ij} \in \mathbb{Z} &
\end{array}
$$

An optimal solution is $x_{11} = x_{22} = x_{33} = 1$ (all others 0), the perfect matching $\{u_1v_1, u_2v_2, u_3v_3\}$.

Bipartite Matching

Interactive demo — open in your browser

Matching Explorer

Interactive demo embedded in this slide of the live lecture, on the bipartite matching problem.

https://ootks.github.io/co250-lectures/demos/MatchingExplorer.html

Nonlinear Programming

Nonlinear Programming

We have covered **linear programming** and **integer programming**.

Nonlinear programming is everything else!

Nonlinear Programming

Arnold and Beth are in a circular room and they want to stand as far apart as possible.

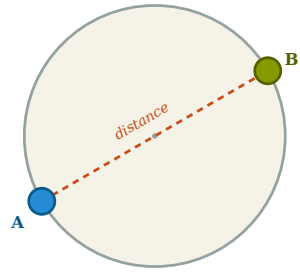
Each person gets to choose a location in the room (x_A, y_A) and (x_B, y_B). Their goal is to maximize the distance between these points:

$$\sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

These choices need to stay in the disk $\{(x, y) : x^2 + y^2 \leq 1\}$.

Nonlinear Programming

$$\begin{array}{ll} \max & \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2} \\ \text{such that} & x_A^2 + y_A^2 \leq 1 \\ & x_B^2 + y_B^2 \leq 1 \end{array}$$



the two points sit at opposite ends of a diameter

Nonlinear Programming

$$\begin{array}{ll} \min & f(x) \\ \text{such that} & g_1(x) \leq 0 \\ & g_2(x) \leq 0 \\ & \dots \\ & g_k(x) \leq 0 \\ & x \in \mathbb{R}^n \end{array}$$

f and each g_i is some (possibly nonlinear) function.

The $\leq$ can also be $\geq$, min can be max, etc. Each change amounts to taking the negative of the objective/constraint function.

Nonlinear Programs: Restaurant Planning Revisited

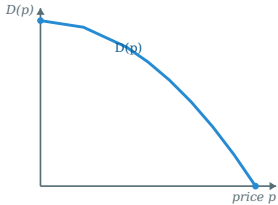
In an earlier lecture, we discussed the planning problem for a restaurant. Let's say that the restaurant also gets to decide their prices.

Let's say that the restaurant needs to just sell one food item, and it wants to determine the price of the food item.

To model this, we need to know the **demand function** that tells us how much food the restaurant sells if it sets the price of that item to p.

Nonlinear Programs: Restaurant Planning Revisited

Suppose that for each $p \in \mathbb{R}_{\geq 0}$, $D(p)$ is how many servings the restaurant can sell if it sets the price of the food to p.



Fewer servings are sold as the price rises.

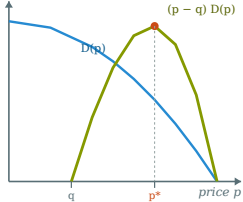
What is the objective here?

Nonlinear Programs: Restaurant Planning Revisited

If the food item costs c to make, then the total profit from selling one food item at price p is $p - c$.

Then the total amount that is sold is $D(p)$, so the total profit is

$$(p - c)D(p).$$

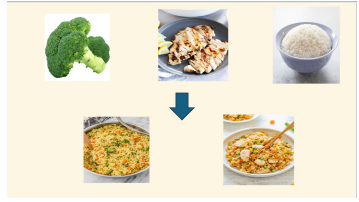


ks at an interior price: too low earns little margin, too high sells

Nonlinear Programs: Restaurant Planning Revisited

Let us now return to the situation in which a restaurant has 2 food items that it can sell, and 3 ingredients with which to make those items.

The restaurant will use **chicken, rice and vegetables** as ingredients to produce either **chicken bowls or vegetable bowls**.



In the earlier lecture, we talked about a multiday planning model, but for now, let's just focus on one day at a time.

Nonlinear Programs: Restaurant Planning Revisited

What are the variables for this model?

r_j = kilograms of ingredient j purchased,
f_i = servings of food i produced,
p_i = price of food i.

Nonlinear Programs: Restaurant Planning Revisited

Objective: Maximize total profit:

$$\sum_i p_i f_i - \sum_j c_j r_j.$$

First term is the total revenue from selling f_i of food i at price p_i, and second term is the cost of buying ingredients.

Nonlinear Programs: Restaurant Planning Revisited

Constraints: Need to buy enough of each ingredient

$$\sum_i n_{ji} f_i \leq r_j$$

Here n_{ji} is how much of ingredient j is needed to make food i.

Also sales limited by demand.

$$f_i \leq D_i(p_i).$$

Here, D_i is the demand function of food i.

Nonlinear Programs: Restaurant Planning Revisited

Overall, the problem is

$$\begin{array}{lll} \text{max} & \sum_i p_i f_i - \sum_j c_j r_j & \\ \text{such that} & \sum_i n_{ji} f_i \leq r_j & \text{for each } j \\ & f_i \leq D_i(p_i) & \text{for each } i \end{array}$$

This problem is nonlinear because the objective involves the term $p_i f_i$, which is quadratic in the decision variables. Also, the constraints involve the possibly nonlinear constraint that $f_i \leq D(p_i)$.

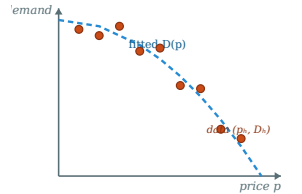
Note that if the p_i are fixed instead of being decision variables, then this is a linear program.

Nonlinear Programs: Restaurant Planning Revisited

What if we don't know the demand function?

Need to use data; say that we look at similar restaurants that have different prices for their food, and how much demand those food items get.

$$\mathcal{D} = \{(p_i, D_i) : i = 1, \ldots, k\}$$



We want to know what the demand is when p is an intermediate value.

Regression Analysis

A basic problem in data science is to find a **function** which fits some data.

There are 3 components to such a regression problem: a function class for the model, the data, and a loss function that measures how well the function fits the dataset.

Classically, we study linear regression (least squares), but modern machine learning is primarily focused on fitting nonlinear functions.

Linear Regression

Given a dataset $\mathcal{D}$,

$$\mathcal{D} = \{(x_i, y_i) : i = 1, \ldots, D\} \subseteq \mathbb{R}^n \times \mathbb{R},$$

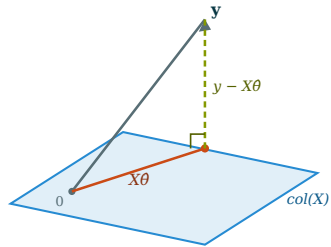
the linear regression problem is to find a linear function of the form $\langle \theta, x \rangle$ that minimizes

$$\sum_{i=1}^{D} (\langle \theta, x_i \rangle - y_i)^2$$

Here, $\theta \in \mathbb{R}^n$.

Linear Regression

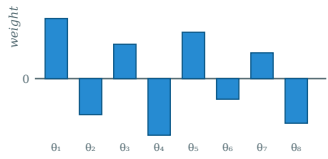
Geometric Interpretation



Least squares projects y orthogonally onto the span of the data.

Sparse Linear Regression

A common issue with linear regression solutions is that they tend to make all of the weights nonzero.



ı least squares typically makes *every* weight nonzero (a dense s

What if we want a good linear model, but one with very few weights being nonzero?

Sparse Linear Regression

Let $\|\theta\|_0$ be the **number of nonzero entries** of the vector. E.g. $\|(1,1,0)\|_0 = 2$.

The sparse linear regression problem is

$$\begin{array}{ll} \text{min} & \sum_{i=1}^{D} (\langle \theta, x_i \rangle - y_i)^2 \\ \text{such that} & \|\theta\|_0 \leq k \end{array}$$

This problem is hard! Requires combinatorial search.

LASSO (constrained formulation)

Let $\|\theta\|_1$ be $\sum_{i=1}^{n} |\theta_i|$. E.g. $\|(1,1,0)\|_1 = 2$.

The (constrained formulation of LASSO) is given by

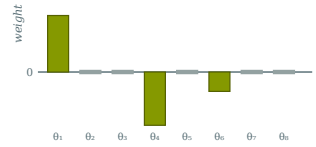
$$\begin{array}{ll} \text{min} & \sum_{i=1}^{D} (\langle \theta, x_i \rangle - y_i)^2 \\ \text{such that} & \|\theta\|_1 \leq k \end{array}$$

LASSO is more often written in its 'penalty formulation', which is just to minimize $\sum_{i=1}^{D} (\langle \theta, x_i \rangle - y_i)^2 + \lambda \|\theta\|_1$ for some $\lambda \geq 0$.

We will talk about how to solve this kind of (convex) problem later.

LASSO (constrained formulation)

Unlike least squares, LASSO tends to set most weights **exactly** to zero, recovering a sparse model.



LASSO drives most weights to *exactly* zero (a sparse solution).

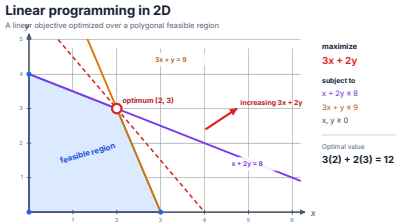
Only a few features (here $\theta_1, \theta_4, \theta_6$) are used; the rest are dropped.

Linear Programming Again

Linear Programming Again

A linear program (LP) is an optimization problem whose objective is a **linear function** and whose feasible set is the set of solutions to some linear inequality.

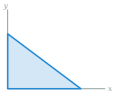
$$\begin{array}{ll} \min & c^T x \\ \text{such that} & Ax \leq b \end{array}$$



Linear Programming Again

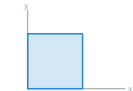
Feasible regions for linear programs are polytopes.

Triangle (2D)



$x \geq 0$
$y \geq 0$
$3x + 4y \leq 12$

Square (2D)



$x \geq 0, \quad x \leq 2$
$y \geq 0, \quad y \leq 2$

Cube (3D)



$0 \leq x \leq 1$
$0 \leq y \leq 1$
$0 \leq z \leq 1$

Tetrahedron (3D)

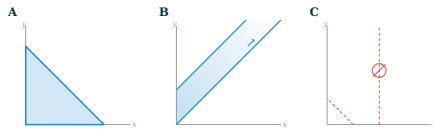


$x \geq 0, \quad y \geq 0, \quad z \geq 0$
$x + y + z \leq 1$

Each polytope is the solution set of a system of linear inequalities $Ax \leq b$.

Linear Programming Again

Polytopes can come in various different shapes, and we can read off various features of the polytope from its linear inequality description.



(1) $x \geq 0$
$y \geq x$
$y \leq x + 2$

(2) $x + y \leq 1$
$x \geq 2$
$y \geq 0$

(3) $x \geq 0$
$y \geq 0$
$x + y \leq 3$

Which polytope corresponds to which system of inequalities?

Answer: A $\leftrightarrow$ (3), B $\leftrightarrow$ (1), C $\leftrightarrow$ (2).

A Coarse Classification of Polytopes

Every polytope comes in one of three shapes: it can be **empty**, it can be **nonempty compact**, or it can be **unbounded**.

A polytope is empty if there are no points in it, e.g.

$$\{(x, y) : x + y \leq 1, x \geq 1, y \geq 1\}.$$

A polytope is unbounded if there are vectors which are arbitrarily long in the set.

$$\{(x, y) : x + y \geq 1, x \geq 1, y \geq 1\}.$$

Note that every vector of the form $(x, 1)$ with $x \geq 1$ is in this set.

A polytope is compact if it is neither empty or unbounded.

We will discuss these different examples in greater detail next lecture.